

AquaSync

A decision-support digital twin for dam–river flood and hydropower optimisation on the Periyar

Track: Climate Resilience & Disaster Preparedness

EVOKE 26 · MACE IoT Club · Mar Athanasius College of Engineering, Kothamangalam

1.0–2.3 m

flood cushion gained driving from
real ensemble forecasts (§4.4)

0.9957

r^2 of the calibrated
level–storage curve

0.30 m

mean error reproducing
observed reservoir level

Rs 0

software and data cost

This document reports a correction to its own brief.

The dataset this project was planned around does not contain the 2018 flood data it was believed to contain. The flagship case study is therefore the **October 2021 Periyar event**, which is fully covered by free public data and is a stronger case. Details in §3 and §11.

Prepared 30 August 2026 · All figures regenerate from public data via `scripts/make_figures.py`

1 · The problem

Kerala's floods are not purely meteorological events. In August 2018, 483 people died and roughly a million were displaced; the subsequent official and academic post-mortems all identified the same aggravating factor — reservoirs held near capacity into an extreme rainfall event, then forced into large simultaneous releases once there was no storage left. The rain was the trigger. The timing of the releases decided how much of it became a disaster.

The reason this keeps happening is a genuine conflict of objectives, not incompetence:

- **The power utility** wants the reservoir full. Head is revenue, and water released without generating is money poured away.
- **Disaster management** wants the reservoir low. Empty storage is the only flood cushion that exists.
- **Release too early** and you have spilled water you needed for the dry season.
- **Release too late** and you must open everything at once, which is precisely the scenario that drowns the towns downstream.
- **The tide** decides whether a release even reaches the sea. At high tide the Arabian Sea holds the lower Periyar up, and the same discharge produces a higher stage upstream.

No operational system in Kerala today optimises these together, on a forecast, across more than one reservoir at a time. That is the gap AquaSync addresses.

2 · The evidence: October 2021, in public data

The argument above is easy to assert and rarely demonstrated. Here it is in the KSEB daily bulletin, downloadable by anyone.

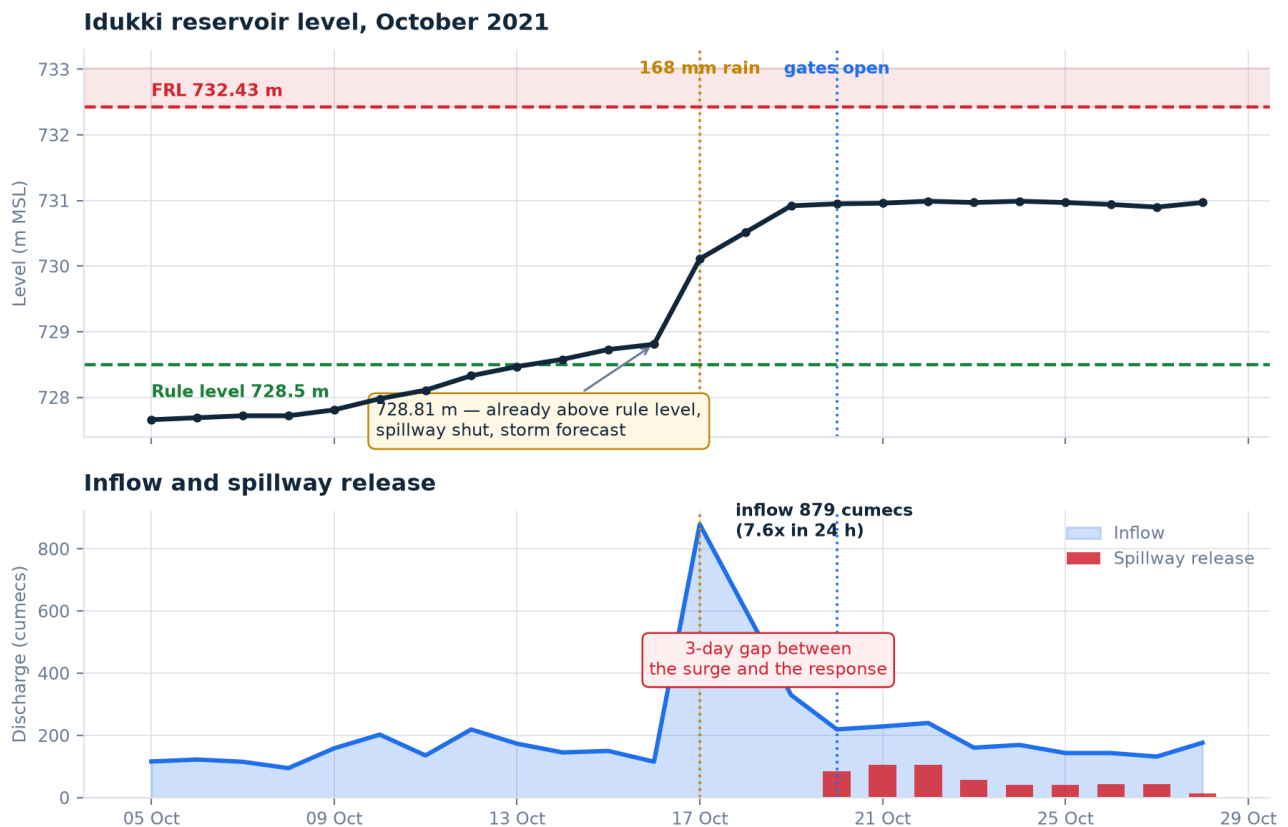


Figure 1 — Idukki, October 2021. The reservoir entered a 168 mm rain day at 728.81 m, already above its own 728.5 m rule level, with the spillway shut. Inflow rose 7.6× in 24 hours to 879 cumecs. The gates opened three days later, at 730.99 m — 1.4 m from FRL. Source: KSEB daily bulletin.

Three days elapsed between the surge and the response. Nothing was concealed and no rule was broken; the level simply rose faster than a reactive procedure can respond to. Every threshold in that chart is published, including the rule level the reservoir was already above when the storm was forecast.

The same thing happened next door, on the same days

Both Periyar reservoirs opened their gates on 20 October 2021 — into the same river

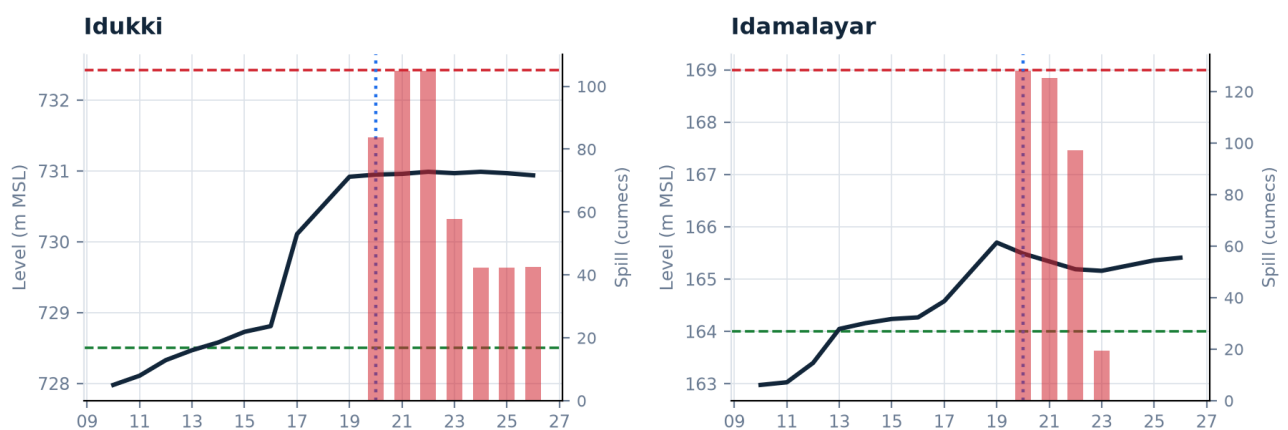


Figure 2 — Idukki and Idamalayar both sat above their rule levels through the storm and both opened their spillways on 20 October 2021 — 84 and 128 cumecs, into the same river, within the same day.

This is the coordination failure, stated precisely.

Two reservoirs jointly control the Periyar. Both absorbed the storm into their freeboard, and both then released into an already-swollen river on the same day. Whether two release pulses superpose into one large peak or spread into two small ones is a **scheduling choice** — and nothing in current practice makes it deliberately.

3 · What AquaSync is

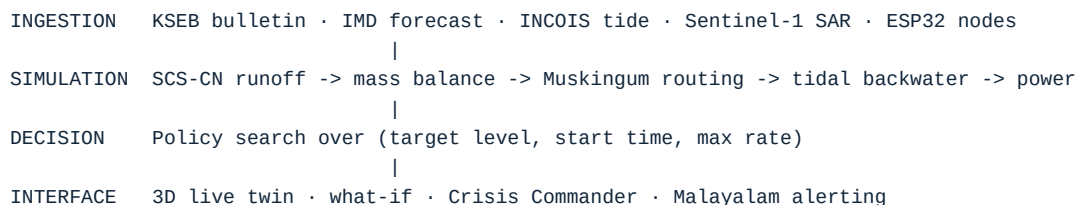
A digital twin of the reservoir–river system that ingests telemetry, rainfall forecasts and tide predictions; simulates the basin forward; searches operating policies; and hands a human operator a specific, implementable instruction.

The output is an instruction, not a dashboard.

“From 06:00 on 10 October, release up to 480 cumecs until Idukki reaches 728.50 m, then hold within ± 0.15 m.”

That is a sentence a control room can execute, a district collector can approve, and an inquiry can later audit. A screen full of gauges is none of those things — and the gap between knowing the level and knowing what to do about it is where the three days in Figure 1 went.

Layer diagram



The five models

#	Model	Answers	Method
1	Rainfall–runoff	How much of the forecast rain reaches the dam, and when?	SCS Curve Number with antecedent-moisture shift, triangular unit hydrograph
2	Reservoir balance	What level results from a given release?	Mass balance on a fitted level–storage power law
3	River routing	What discharge arrives at Aluva, and at what hour?	Muskingum storage routing, auto sub-reached for stability
4	Tidal backwater	How much can the river safely carry right now?	Harmonic tide + exponential backwater decay, giving effective conveyance
5	Hydropower	What does this release cost or earn?	Density x gravity x flow x head x efficiency, with a turbine hill diagram and time-of-day tariff

An asymmetry specific to Idukki is worth knowing, because it changes the whole framing: the Moolamattom powerhouse discharges into the **Muvattupuzha**, not the Periyar. Generation therefore does not load the Periyar at all — only the spillway does. Sending water through the turbines is simultaneously the profitable option and the one that spares Aluva. Power and safety are not inherently opposed. Only *timing* puts them in opposition.

4 · Results

4.1 · The model is calibrated, and the data needed cleaning first

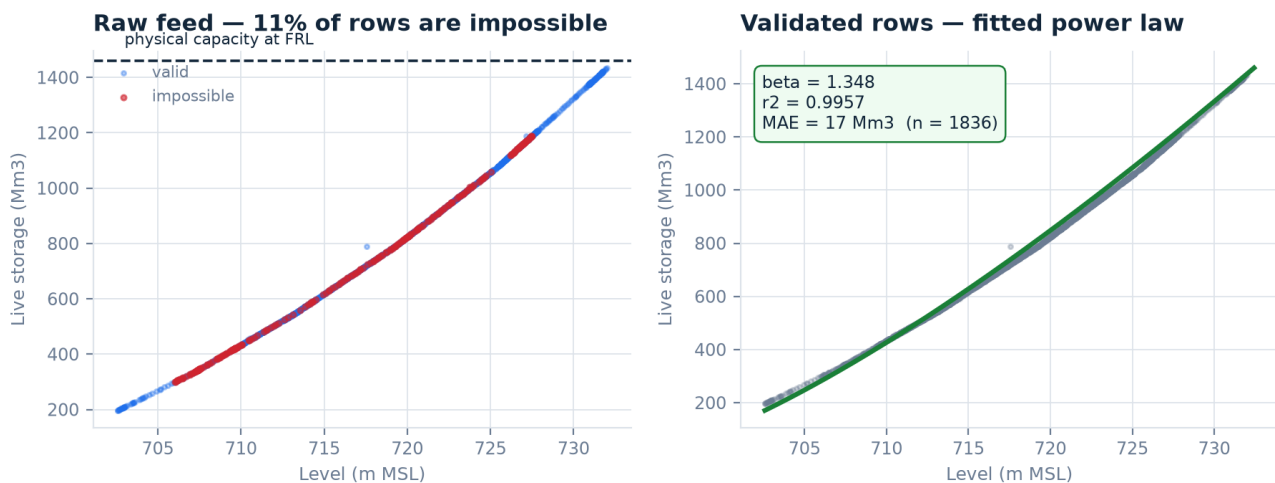


Figure 3 — About 11% of the source dataset is physically impossible (left): live storage above the reservoir's stated capacity, storage percentages over 1,000%. Fitted on the 1,836 validated rows (right), the level–storage power law fits an exponent of 1.348, $r^2 = 0.9957$, MAE 17 Mm³.

Fitting the same curve on the uncleaned feed gives $r^2 = 0.784$ and an MAE of 174 Mm³ — the corrupt block alone displaces the curve by more than the entire flood cushion being modelled. Data validation was not housekeeping here; it was the difference between a usable model and a confidently wrong one.

4.2 · Replaying October 2021, and an episode the model never saw

Fed the releases that actually occurred, the twin reproduces the observed Idukki level with a mean error of **0.30 m** (maximum 0.57 m) over 20 days. That agreement is what earns the right to ask a counterfactual question at all.

Reproducing one episode is fitting; reproducing a second one the model was never shown is validating. Replayed over 1–20 August 2022 — an episode nothing was tuned on — the error is **0.32 m**, within 5% of October 2021's. One wrinkle worth stating: the drift reverses between the two episodes (October 2021 ends +0.52 m, August 2022 -0.27 m), which argues against a single missing loss term and points at event-specific interpolation timing instead.

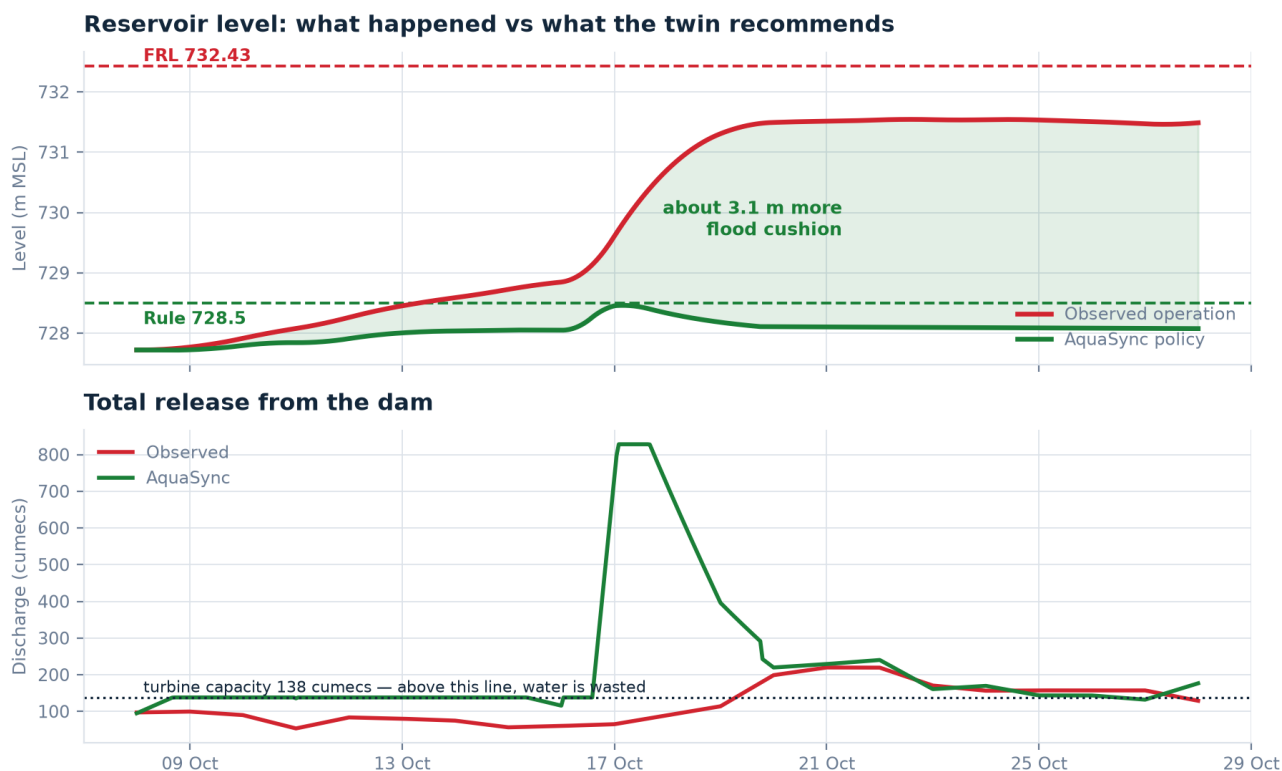


Figure 4 — Observed operation (red) against the policy AquaSync selects (green). The policy ends the episode roughly 3.1 m lower, holding 4.0 m of flood cushion instead of 0.9 m.

4.3 · What forecast lead time actually buys

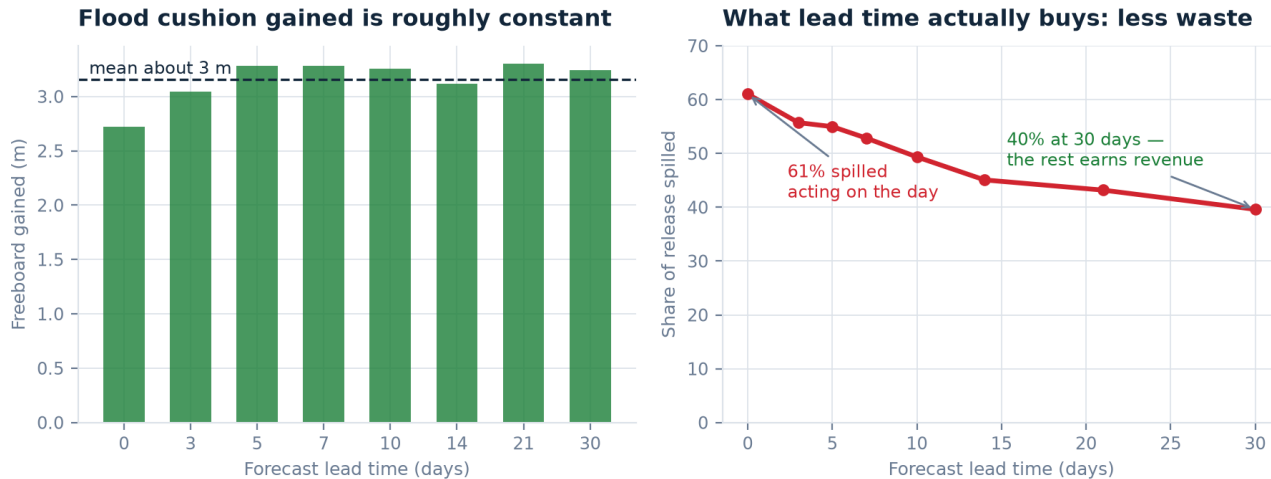


Figure 5 — The flood cushion gained is roughly constant across lead times (left). What lead time changes is waste: the share of released water that goes over the spillway instead of through the turbines falls from 61% to 40%.

The headline finding

Over this episode, a policy-based release schedule delivers about **3 m more flood cushion** than what actually happened while generating **marginally more revenue** (Rs 4–10 crore), because it moves the same volume of generation into higher-tariff hours.

The safety-versus-power trade-off everyone assumes is largely an artefact of *hoarding reservoir level* rather than *scheduling releases*. Lead time does not buy the cushion — it buys the ability to take the cushion through the turbines instead of over the spillway.

One further result is worth stating because nobody set it: given only the physics and the objective, the optimiser converged on a target level of **728.43 m** at nearly every lead time tested — within 7 cm of the **728.50 m** that KSEB's own 2020 rule curve prescribes for 31 August.

But “just follow the rule curve” is NOT the recommendation, and the audit record says so.

It is tempting to read the previous paragraph as vindication of the published rule curve. The CAG performance audit of Kerala's flood preparedness rules that out. Reading its Tables 3.6 and 3.7 directly: actual Idukki spills over 14–18 August 2018 totalled **467.51 MCM**, while the 2020 rule curve would have required **531.03 MCM** across the same window.

On the worst days of the worst flood in a century, mechanical compliance with the published curve would have put *more* water into the Periyar, not less. The agreement at 728.50 m is a coincidence of date — 31 August is the curve's most permissive step — not a validation of it.

What the result actually supports is narrower and more defensible: **a forecast-driven target, recomputed as conditions change, lands near sensible operating levels without being told what they are.** The value is in the recomputation, not in the number.

This correction came from the project's own research sweep rather than from a reviewer, which is the outcome to aim for. The full evidence, including the positions that contradict this project's thesis, is in the companion *Deep Research Report*.

4.4 · What survives once the forecast is real

Every number above hands the optimiser the inflow that actually occurred. No operator has that. This study re-runs the same October 2021 decision from a 30-member NOAA GEFS rainfall ensemble issued *before* the storm: each member is bias-corrected against IMD gridded rainfall, pushed through the same SCS-CN and Muskingum chain, and given its own policy search. One policy is then committed to under that uncertainty and scored against what actually happened.

Lead time	Ensemble issued	Expected value	Minimax regret
24 h	2021-10-15 18z	0.81 m (26%)	1.03 m (33%)
90 h	2021-10-13 00z	1.03 m (33%)	2.30 m (74%)
120 h	2021-10-11 18z	1.03 m (33%)	1.07 m (34%)

Percentages are the share of the 3.11 m that perfect foresight buys over the same window. Expected value commits to the candidate policy with the lowest mean cost across all 30 members; minimax regret commits to the one whose worst case across the ensemble is least bad.

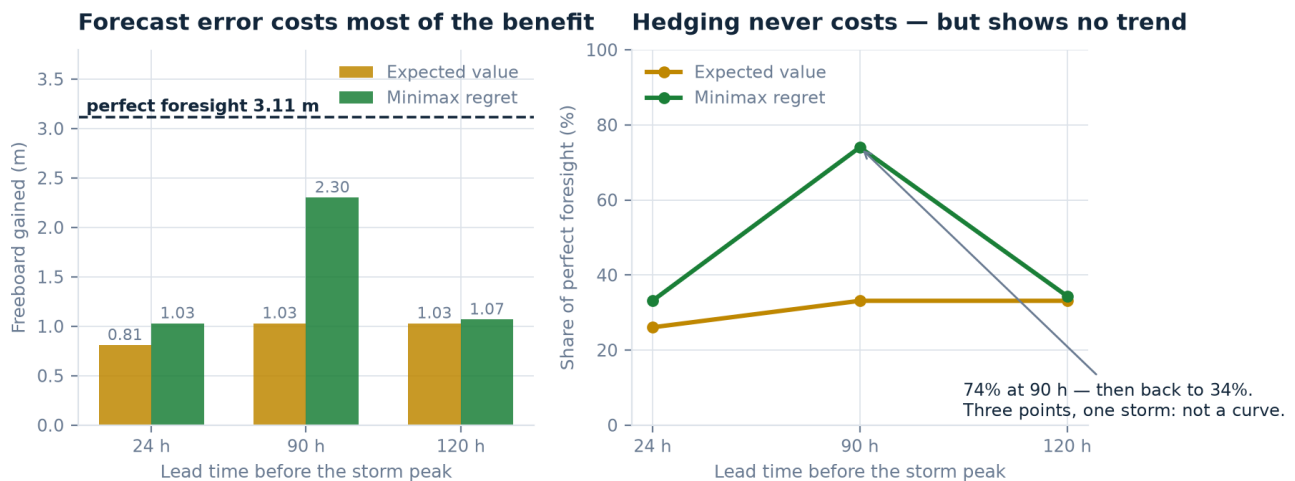


Figure 6 — The benefit that survives a real forecast, by lead time and decision rule, against the perfect-foresight ceiling.

Between a quarter and three quarters of the benefit survives — and the choice of decision rule matters as much as the lead time.

Driven from a real ensemble the policy retains **26–74%** of the perfect-foresight gain. **Hedging against the worst ensemble member never did worse than betting on the average one, and at 90 h it did more than twice as well (74% against 33%).** That makes minimax regret a safe default: it costs nothing on the occasions it does not help. The perfect-foresight figures elsewhere in this document are a ceiling, and should be read as one.

What did not survive the third data point.

After the 24 h and 90 h runs it looked like the minimax advantage grows with lead time — a 7-point gap widening to 41, and mechanistically plausible, since more lead time means more ensemble spread and more to gain from hedging. **The 120 h run breaks it:** the gap collapses to about a point. **Three points from one storm are not a decay curve.** The likely confound is that the bias correction is a single-event multiplicative factor and it swings between runs (1.68× to 3.18×), which alone can move which member the optimiser treats as its worst case. The claim is withdrawn rather than defended, and recorded here because a retraction found internally is worth more than one found by a reviewer.

4.5 · Two dams, and a result that argues against the obvious extension

Everywhere else in this project the two Periyar reservoirs are optimised one at a time. Since the evidence for the problem is that they failed *jointly* (Figure 2), scheduling them together looked like the single most valuable thing left to add. Routing both releases through the river DAG to their shared confluence over the 481-hour October 2021 window says otherwise.

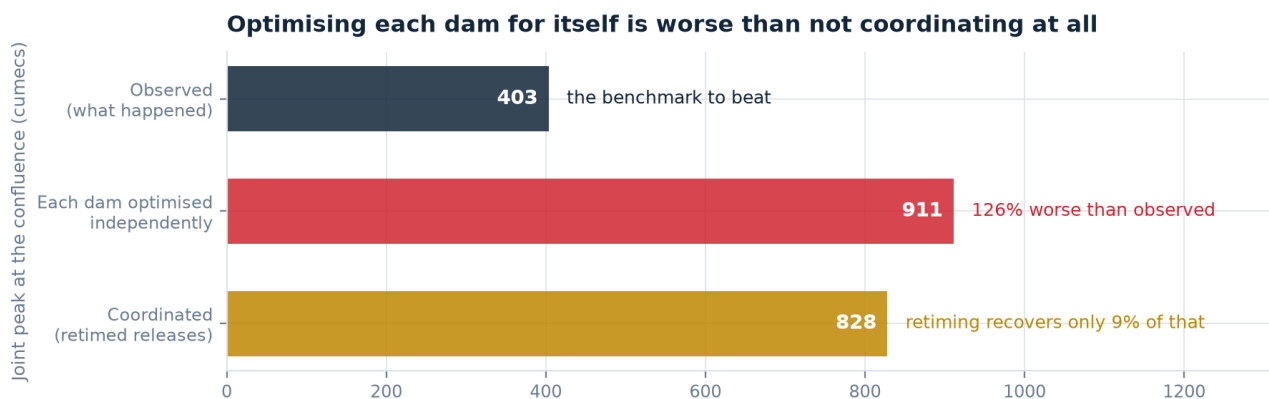


Figure 7 — Joint peak at the confluence under three regimes. These are the two dams' contribution only: the ungauged lateral inflow between the dams and Aluva is not included, so the absolute values are not total river discharge and must not be read against bankfull. The comparison between the three regimes is valid; the cumec values on their own are not a flood-risk verdict.

Optimising each dam for itself is not a neutral simplification — it is actively worse than the uncoordinated baseline.

Each dam optimising alone raises the joint peak to **911 cumecs** against the **403** that actually occurred — **126% worse**. Idukki's own optimum releases at the top of its rate grid because, scored against the downstream reach *alone*, that looks entirely safe. It is not safe once Idamalayar's simultaneous release arrives at the same confluence — a combination neither dam's objective function can see.

And retiming does not fix it. Sweeping both dams' start hours — the most direct reading of “make the pulses not superpose” — recovers only 9% of the gap it opened (911 to 828 cumecs), nowhere near the observed 403. Once each dam is already optimised for its own safety at maximum rate, this is not a timing problem; it is an objective-function problem. The correctly-scoped next piece of work is a genuinely joint objective — each dam's policy scored against the actual combined downstream discharge rather than its own reach in isolation — not a bigger timing search. That is now the largest open modelling item in the project, and §9 records it as such.

5 · What is genuinely new here

	Common approach	AquaSync
Output	A dashboard showing the current level	An implementable three-parameter release policy
Timing	Reactive — alerts once thresholds are crossed	Prescriptive — acts on a forecast, before the surge
Scope	One reservoir in isolation	Both Periyar dams routed to their shared confluence — which is how this project found that optimising them separately makes the joint peak worse (\$4.5)
Objectives	Water level only	Flood, dam safety, generation revenue and gate wear, jointly weighted
The sea	Ignored	Tidal conveyance windows treated as a schedulable resource
Validation	Synthetic or hand-picked data	Replay against real telemetry, with the error reported
Data trust	Feed consumed as-is	Validation layer; 11% of the source found to be corrupt
Failure	Cloud dashboard, dark when the network dies	LoRa fallback, local logging, tamper-evident hash chain

The three ideas that carry the project

1 · Search policies, not schedules. The first implementation searched over hundreds of independent hourly release values and produced results that were non-monotonic in lead time — 10 days looked better than 14, which looked better than 21. That was not a finding, it was a search artefact: a longer horizon has more free variables, so a fixed budget covers it more sparsely. Reformulating the search over (*target level*, *start time*, *max rate*) made the space small enough to enumerate exhaustively — deterministic, reproducible, monotonic, and expressible as a sentence an operator can act on.

2 · Price the trade-off honestly, then discover it is smaller than assumed. Only the portion of a spill the turbines could actually have absorbed counts as forgone generation; water above turbine rating had to be spilled regardless. Getting this wrong overstates the cost of acting and biases an operator toward holding water. Counting it correctly is what reveals that the conflict is mostly about timing.

3 · Treat the tide as a schedulable resource. Cochin's spring tidal range is only about a metre, but on a river already at bankfull a metre decides whether a town floods. It is also perfectly predictable and recurs twice a day, which makes it a free, repeating window in which the same volume can be moved at materially lower risk.

6 · How it gets built

Six phases. Each ends in something demonstrable, so the project is presentable at any point after Phase 2 rather than only when complete.

Phase	Weeks	Deliverable	Done when
0 · Foundation	0.5	Repo, data pipeline, validation layer	Idukki and Idamalayar load clean; quality report prints
1 · Twin core	1.5	Mass balance, level–storage calibration, replay	Observed October 2021 level reproduced to under 0.5 m
2 · Routing & tide	1.5	Muskingum reaches, harmonic tide, conveyance	Downstream hydrograph at Aluva with a stated travel time

Phase	Weeks	Deliverable	Done when
3 · Decision engine	1.5	Policy search, objective weights, baseline comparison	A counterfactual with a defensible headline number
4 · Hardware rig	2	Two-tank HIL bench, ESP32, stepper gate, telemetry	Twin drives the physical gate; fault injection recovers
5 · Interface	1.5	3D live twin, what-if panel, Crisis Commander	A stranger can run the demo unaided
6 · Hardening	1	Tests, docs, rehearsal, poster, offline fallback	Full demo runs with the network cable pulled

The failure mode to design against is feature creep, not lack of ambition.

The planning material behind this project accumulated twenty-five candidate upgrades — satellite calibration, agent-based evacuation, graph neural routing, post-quantum SCADA cryptography, an autonomous bathymetry boat. Each is individually sound. Attempting more than two produces a table of half-working prototypes and no time to rehearse, which is the single most common way a strong idea loses.

Phases 0–3 plus a V1 rig is a complete, winning project. Everything else is backlog.

7 · Components and cost

Four tiers. **V1 alone is a complete demonstration.** Prices are indicative INR at August 2026, from Indian vendors (Robu.in, ThinkRobotics, ElectronicsComp).

Tier	What it adds	Time	Rs	Cumulative
Tools	Iron, solder, multimeter, strippers — if not owned	—	1,480	1,480
V1	Two-tank HIL rig: ESP32, JSN-SR04T, DS18B20, NEMA 17 gate, YF-S201 flow, pump, acrylic	1 wk	6,250	7,730
V2	Off-grid: LoRa SX1278 (433 MHz), SD logging, INA219 jam detection, solar	1 wk	2,300	10,030
V3	Edge AI: ESP32-S3, ESP32-CAM, 24 GHz radar, hydrostatic transmitter, I ² S mic	2 wk	5,900	15,930
V4	Raspberry Pi offline command post, GPS bathymetry boat	3 wk	9,900	25,830

Key V1 components

Component	Specification	Rs	Why this part
ESP32-WROOM-32	38-pin DevKit, 240 MHz, Wi-Fi + BT	450	38-pin, not 30 — you need the extra ADC pins
JSN-SR04T v3.0	Waterproof ultrasonic, 25–450 cm, ±1 cm	550	Separate transducer on a cable; survives splash
DS18B20	1-Wire waterproof probe, ±0.5 °C	180	Not optional — sound speed drifts 0.6 m/s per °C
NEMA 17 + A4988	42 mm, 1.8°, 4.2 kg·cm, 1/16 microstep	780	Precise, repeatable sluice gate positioning
YF-S201	Hall flow, 1–30 L/min, G1/2 in	350	Closes the mass balance loop
BMP280	Barometric, ±0.12 hPa	150	Pressure-drop squall pre-detection
Limit switches	SPDT lever × 2	80	A stepper has no position feedback; these give it a datum

Two wiring traps worth knowing before you order.

Ultrasonic echo pins output **5 V** and the ESP32 is 3.3 V tolerant only — a divider on every echo line is mandatory, not optional. And GPIO 34/35 are **input-only with no internal pull-ups**, so limit switches wired there need external ones.

Note also that the component links in the material this project was planned from were unreliable — one Amazon ASIN appeared as a breadboard, a jumper set, a DHT22, a resistor kit and a multimeter. The bill of materials is anchored on part numbers and specifications instead, which is what actually survives.

8 · Data and open-source foundations

Everything is free. Nothing requires an institutional licence.

Source	Provides	Access	Verified
amith-vp/Kerala-Dam-Water-Levels	Daily level, storage, inflow, powerhouse and spillway discharge, rainfall for 18 KSEB reservoirs, with FRL/MWL/rule/alert thresholds	Raw JSON on GitHub, updated daily	Yes — Aug 2020 to Aug 2026
IMD gridded rainfall	0.25° daily rainfall, 1901–present	Free download	—
Open-Meteo / OpenWeatherMap	72–120 h rainfall forecast	Free API tier	—
INCOIS	Tide predictions for Kochi	Free for Indian academic use	—
ISRO Bhuvan CartoDEM	10–30 m terrain for inundation mapping	Registration	—
Sentinel-1 SAR	Cloud-penetrating flood extent	Google Earth Engine / Copernicus	—
OpenData Kerala	Ward and panchayat boundaries (GeoJSON)	GitHub	—

Software stack

Python 3.12, NumPy, SciPy, pandas for the twin; FastAPI and WebSockets for telemetry; Three.js for the 3D interface; PlatformIO and Arduino-ESP32 for firmware; Matplotlib and ReportLab for this document. Total licence cost Rs 0.

Deliberately not used

Considered	Why not
RTC-Tools (Deltares)	Genuinely industry-grade reservoir optimisation, but needs a CasADi/IPOPT toolchain and the problem here is small enough that a transparent exhaustive search is both sufficient and far easier to defend in a five-minute Q&A
Google Flood Forecasting API	Excellent, but it is a forecast <i>product</i> . Using it as ground truth for our own forecast would be circular. Kept as a benchmark to compare against
Kaggle “Kerala Floods 2018”	District casualty and rainfall totals only — no reservoir telemetry, so it cannot validate a routing model
LISFLOOD-FP 2D inundation	The right long-term answer for street-level depth mapping, but it needs a calibrated DEM and roughness field. Roadmap, not scope

9 · Limitations

Stated plainly, because every one of these will be found by anyone who looks, and finding them first is the difference between a limitation and a hole.

Limitation	Consequence	Mitigation
Headline figures assume perfect foresight	The optimiser sees the true inflow when choosing a policy, so every perfect-foresight number here is a ceiling . Driven from a real 30-member ensemble the policy retains 26–74% of it (§4.4)	Measured rather than assumed: quote the forecast-driven range for any operational claim. Remaining gap: one storm, three lead times
Daily input resolution	Bulletin data is one reading per day, interpolated to hourly. Sub-daily peaks are smoothed away; peak timing carries roughly ± 12 h	CWC 15-minute telemetry; ESP32 nodes for local sub-daily truth
Routing parameters are anchored, not calibrated	K and x are anchored to CWC's published 8-hour Idukki–Neeleeswaram travel time. A direct fit against 1,967 days of gauge data failed ($r^2 = 0.005$): daily data is $\sim 3\times$ coarser than the signal. Downstream discharge is indicative, not measured	CWC 15-minute telemetry — the highest-value data upgrade. Until then, never quote Aluva discharge as measured
Replay error 0.30 m MAE	Roughly 0.5 m of the ~ 3 m freeboard claim sits inside model error	Quote it as “about 3 m”, never to two decimals
No 2D inundation	The twin gives river discharge, not which streets flood	LISFLOOD-FP or a HEC-RAS coupling on a Bhuvan DEM
Each dam is optimised against its own reach alone	Not merely incomplete — measured as harmful: optimising the two dams independently raises the joint peak at their confluence 126% above what actually happened, and retiming recovers only 9% (§4.5)	A joint objective: score each dam's policy on combined downstream discharge, not its own reach. The largest open modelling item
No lateral inflow between the dams and Aluva	The cascade figures are the two dams' contribution only, not total river discharge, and must not be read against bankfull thresholds	RiverNetwork already accepts lateral inflows; no tributary or local-runoff series exists in the project's data holdings yet
Indicative tariffs	Revenue figures are order-of-magnitude, not audited	Substitute the current KSERC order before quoting rupees to a utility
Institutional adoption	A utility will not accept an external recommendation engine on trust	Shadow mode: run alongside real operations for a full monsoon, log what it would have advised, publish the comparison

9.1 · Evidence against this project's thesis

A deep-research sweep run against this project found five positions that complicate or contradict its argument. They belong here rather than in a footnote, because the first domain-literate reviewer will find them anyway.

Challenge	What it means for AquaSync
The literature does not agree that dams worsened the 2018 flood. CWC's own September-2018 study concludes the reservoirs <i>attenuated</i> the peak — Idukki 2,532 down to 1,500 cumec, about 41%.	The framing “dam mismanagement caused the flood” is contested by the agency that owns the data. State the conflict; do not assert one side.

Challenge	What it means for AquaSync
The most-quoted “reservoirs made it worse” study was rejected. Mishra et al.’s HESS preprint carries “not accepted for further review”, and its headline number differs from the peer-reviewed figure by about 3.5×.	Do not cite it. Its absence from the argument is a strength.
The forecast skill this approach assumes may not exist in this basin. Sudheer et al. report that probability of detection for rainfall above 25 mm/day at 3–5 day range is poor <i>even with ensembles</i> .	This is the deepest problem in the project. It is not solved by better optimisation, and it bounds what any forecast-driven system can deliver here.
Even a perfect optimiser has a modest ceiling. The same authors put achievable peak attenuation from advance emptying at 16–21% , and find downstream flows become insensitive to reservoir state below 50% storage.	A useful sanity bound. Any claim materially above it needs extraordinary evidence.
FIRO took about a decade of shadow operation to change one water control manual that had been revised twice in 66 years.	The adoption timeline in this document is optimistic. Shadow mode is not a phase; it is most of the work.

The honest summary of all five.

AquaSync is a decision-support tool whose upper bound is set by monsoon forecast skill over the Western Ghats, operating in a domain where the causal claim it rests on is genuinely disputed. That is a smaller claim than the one this document opened with, and it is the one the evidence supports.

The system is advisory, permanently.

AquaSync never operates a gate. If an automated system opens a spillway and someone drowns, the question of who is accountable has no acceptable answer; a recommendation approved by a named officer has a clear one. The hardware demonstrator does close the loop onto a servo, because it is a 30 cm acrylic tank — and that distinction should be said out loud during the demo rather than left for someone to catch.

10 · Roadmap beyond the expo

Horizon	Work	Why it matters
Now – expo	Joint cascade objective (\$4.5); V1 rig; live backend and Crisis Commander mode	The forecast-error and cascade studies are done and reported in \$4; what remains is acting on what they found
3–6 months	CWC 15-min telemetry; 2D inundation on Bhuvan DEM; Sentinel-1 extent validation; field-deploy one ESP32 node	Moves from daily hindcast to operational nowcast
6–12 months	Shadow-mode trial with KSEB/KSDMA; Malayalam last-mile alerting via LDMC and Kudumbashree networks; OGC SensorThings output	Builds the operational trust record adoption requires
Beyond	All Kerala cascades; ensemble and reinforcement-learning policies; dam-breach mode; public tamper-evident release ledger	From one basin to a statewide decision layer

11 · Two corrections to the source material

This project was planned from a long set of prior design conversations, archived in `reference/source_chats/`. Two of their load-bearing assumptions did not survive contact with the data, and both are recorded here so the same ground is not covered twice.

1 · The dataset does not contain 2018 data.

The planning material identified `amith-vp/Kerala-Dam-Water-Levels` as “*exactly the 2018 Idukki dam data you need*” and built the entire flagship demo — a “2018 replay” — on it. The historical files begin on **13 August 2020**. There is no 2018 record in the repository at all.

The flagship case study moved to **October 2021**, which is fully covered, complete in every field, and a better demonstration. The 2018 flood remains the reason anyone cares — but no quantitative claim now depends on it. Acquisition routes for genuine 2018 data are documented in `docs/data-sources.md`.

2 · About 11% of the dataset is corrupt.

Rows between 2020-09-25 and 2021-04-30 report live storage above the reservoir's physical capacity and storage percentages over 1,000% — consistent with a column-alignment bug upstream. Using the feed unvalidated degrades the level–storage calibration from r^2 0.996 to 0.784.

12 · The two-minute pitch

“In October 2021, the Idukki reservoir entered a 168 millimetre rain day already *above* its own published rule level, with the spillway shut. Inflow rose seven-and-a-half times in twenty-four hours. The gates opened three days later — and Idamalayar, on the same river, opened its gates on exactly the same day. Everything I just said is in the public KSEB bulletin. Nobody hid anything, and nobody broke a rule.

AquaSync is a digital twin of that system. It simulates the catchment, the reservoir, the river and the tide, and searches for the release policy that best trades off flood risk, dam safety and generation revenue. It reproduces the observed October 2021 level to within thirty centimetres, which is what earns it the right to ask: what should have been done instead?

The answer is about three metres more flood cushion — with *slightly more* revenue, not less, because the same water goes through the turbines at better hours instead of over the spillway. The safety-versus-power conflict everyone assumes is mostly an artefact of acting late.

And the operating level the optimiser chose? 728.43 metres — within seven centimetres of what KSEB's own rule curve prescribes for that date, without ever being told it. I want to be careful here, because the CAG audit shows that mechanically following that curve through August 2018 would actually have released *more* water, not less. So the point is not that the rule is right. The point is that a system recomputing the target against a live forecast arrives somewhere sensible on its own — and keeps arriving there as conditions change, which a fixed curve cannot do.”

Demonstration sequence

	Beat	Duration
1	The physical rig is already running: pump on, gate holding level. Point at it, do not explain it yet	—
2	Figure 1 on screen. “This is real, and it is public.”	30 s
3	Dump water into the upstream tank — a live storm. The twin sees inflow rise and opens the gate <i>before</i> the reservoir tank reaches its FRL line	40 s
4	Flip the 'sensor failure' switch. The twin detects the disagreement, falls back to mass-balance state estimation, and keeps controlling	30 s
5	Hand over the tablet: “You are the operator. It is 16 October 2021.” Let them try to save Aluva, then show them the optimiser's answer	40 s
6	Pull the network cable. Everything keeps running on LoRa and the local Pi	20 s

Beat 4 is the one that wins the room. Anyone can demonstrate a system working; demonstrating a system *failing correctly* is what convinces an engineer that it was built by someone who expected it to be used.

Repository · `PROGRESS.md` for live status, `ROADMAP.md` for sequencing, `ACTION_PLAN.md` for the next fourteen days. All figures and every number in this document regenerate from public data with `python scripts/make_figures.py`.